

Auto Email / Ticket Categorizer

Lightweight NLP classifier that reads an incoming support ticket (subject + body) and routes it to Billing, Technical, HR, or General — with confidence scores, a human-review threshold, and simple priority tagging.

STACK

Python · scikit-learn

FEATURES

TF-IDF + Logistic Regression

BONUS

Confidence · Review · Priority · CLI

1. Quick start

From the project root, create a virtualenv and run:

```
# create / use a virtualenv
python -m venv venv

# Windows
venv\Scripts\python.exe -m pip install -r requirements.txt
venv\Scripts\python.exe src\train.py
venv\Scripts\python.exe notebooks\ticket_categorizer_demo.py
venv\Scripts\python.exe src\predict.py --interactive

# macOS / Linux
venv/bin/python -m pip install -r requirements.txt
venv/bin/python src/train.py
venv/bin/python notebooks\ticket_categorizer_demo.py
venv/bin/python src/predict.py --interactive
```

train.py prints holdout accuracy, a classification report, and a confusion matrix, then saves a full-data model to models/.

2. What it does

Step	Implementation
Clean text	Lowercase, strip HTML/URLs/emails/punctuation/stopwords (src/preprocess.py)
Features	TF-IDF unigrams + bigrams on combined subject + body
Model	Logistic Regression (sklearn)
Evaluate	Stratified holdout → accuracy, precision/recall/F1, confusion matrix
Infer	predict_ticket(subject, body) for one ticket on demand

Step	Implementation
Bonus	Confidence %, review if confidence < 60%, urgent/normal keywords, live CLI

Categories: #billing #technical #hr #general

3. Project layout

```

├─ data/tickets.csv          # labeled training tickets
├─ src/
│   ├── preprocess.py        # text cleaning
│   ├── train.py             # train + evaluate + save model
│   └── predict.py           # inference API + interactive CLI
├─ notebooks/
│   └── ticket_categorizer_demo.py # 5 sample predictions
├─ models/                   # created by train.py (gitignored)
├─ requirements.txt
└─ README.md

```

4. Example output

```

SUBJECT: Invoice missing
-> label=Billing, confidence=0.70, review=False, priority=urgent
-----
SUBJECT: App crashes on save
-> label=Technical, confidence=0.73, review=False, priority=normal
-----
SUBJECT: Leave balance query
-> label=HR, confidence=0.67, review=False, priority=normal
-----
SUBJECT: Refund status
-> label=Billing, confidence=0.52, review=True, priority=normal
-----
SUBJECT: Password reset
-> label=Technical, confidence=0.73, review=False, priority=normal

```

Ambiguous or low-confidence tickets stay labeled with the model's best guess but are flagged `needs_human_review=True` so they can go to a manual queue instead of auto-assign.

Interactive CLI:

```
python src/predict.py --interactive
```

```

Subject: Production outage
Body   : Service is down and not working – urgent
category : Technical
confidence: 76.1%
human review: no (auto-assign)
priority : urgent

```

5. Approach summary

Cleaned subject+body text, vectorized with TF-IDF (1–2 grams), and trained Logistic Regression for fast short-text classification with usable probability estimates. Predictions return the category plus a confidence score; below ~60% confidence the ticket is flagged for human review rather than auto-routed. Priority (urgent / normal) is a lightweight keyword layer (urgent, down, asap, not working, critical, outage).

6. Reflection

With more real helpdesk data I would expand coverage of ambiguous General vs Technical tickets and rare failure phrases. Next I'd calibrate probabilities, compare linear SVM / Multinomial Naive Bayes, and learn priority instead of hard-coded keywords. A human-in-the-loop queue for low-confidence tickets, plus periodic retraining on reviewed labels, would make routing more reliable in production.

7. Requirements

- Python 3.9+
- scikit-learn, pandas, joblib (see requirements.txt)